

Replicating Chen, Yu & Lai (2006): A Step-by-Step Tutorial on Modeling Particle Distribution and Deposition in Indoor Environments with a Drift-Flux Model

Replication Study — OpenFOAM + Python

May 15, 2026

Abstract

This tutorial provides a complete, reproducible walkthrough for replicating the computational study of Chen, Yu & Lai (2006), “Modeling particle distribution and deposition in indoor environments with a new drift-flux model” (*Atmospheric Environment* 40, 357–367). We cover all steps from geometry definition and mesh generation in OpenFOAM, steady-state airflow simulation with the RNG k - ε turbulence model, extraction of velocity fields, and a custom Python solver for particle transport using the drift-flux formulation with Lai & Nazaroff (2000) wall deposition. Two ventilation cases are presented: a low-flow case ($U = 0.225 \text{ m s}^{-1}$, ACH= 10) and a high-flow case ($U = 0.45 \text{ m s}^{-1}$, ACH= 20). All numerical data, code, and configuration files are available in the accompanying replication package.

Contents

1	Introduction	3
1.1	What We Replicate	3
1.2	Why This Matters	3
2	Mathematical Model	4
2.1	RANS Airflow Equations	4
2.2	Drift-Flux Particle Transport Equation	4
2.3	Slip Correction and Settling Velocity	5
2.4	Brownian Diffusion	5
2.5	Lai & Nazaroff (2000) Wall Deposition Model	5
3	Geometry and Mesh	6
3.1	Multi-Block Meshing Strategy	6

4	OpenFOAM Configuration	12
4.1	Turbulence Model Setup	12
4.2	Transport Properties	13
4.3	Time Control and Solver Monitoring	14
4.4	Discretization Schemes	16
4.5	Linear Solver Settings	17
4.6	Boundary Conditions	19
4.7	Initial Condition Files	19
5	Running the Airflow Simulation	26
5.1	Convergence Monitoring	26
6	Extracting Fields to Python	26
6.1	The Cell Ordering Challenge	26
6.2	Reading OpenFOAM Fields	27
7	Particle Transport Solver	28
7.1	Solver Design: v2 with Mass Conservation Fix	28
7.2	Discretization Details	28
7.3	Vectorization Strategy	29
8	Results and Validation	30
8.1	Airflow Field	30
8.2	Concentration Evolution	32
8.3	Mixing Dynamics	32
8.4	Particle Transport Results	34
8.5	Key Observations	34
9	HPC Exercise: Running on ALCF Systems	35
9.1	Preparing the Case (Common Steps)	36
9.2	Case A: Aurora (Intel GPU Cluster)	36
9.3	Case B: Polaris (NVIDIA A100 Cluster)	37
9.4	Case C: chiatta00 (JLSE Interactive Node)	39
9.5	Cross-Platform Comparison Exercise	40
10	Discussion and Limitations	41
11	Conclusion	41

1 Introduction

Indoor air quality (IAQ) directly affects human health and comfort. In mechanically ventilated spaces, particles released from sources (e.g., cooking, cleaning, occupant activity) disperse and deposit on surfaces. Understanding this process is essential for exposure assessment, contaminant control, and ventilation design.

Chen et al. (2006) proposed a drift-flux model that couples the Reynolds-Averaged Navier-Stokes (RANS) airflow solution with a scalar transport equation for particle concentration. The key innovation is a semi-empirical wall deposition velocity model based on the near-wall resistance integral approach of Lai & Nazaroff (2000), combined with the Cunningham slip correction and Stokes settling velocity. This avoids the high computational cost of Lagrangian tracking while capturing size-dependent deposition.

1.1 What We Replicate

This tutorial replicates the computational methodology and results of Chen et al. (2006) for a single-zone ventilated room.

Important note: The geometry used in this study is a *laboratory-scale model chamber*, not a full-size room. The chamber dimensions are $0.8 \times 0.4 \times 0.4$ m — roughly the size of a large aquarium. At approximately 1:7 scale, this corresponds to a full-size room of about $5.6 \times 2.8 \times 2.8$ m ($\sim 18 \times 9 \times 9$ ft). Scale modeling is standard practice in ventilation research because it dramatically reduces the cost and complexity of validation experiments (e.g., Phase Doppler Anemometry measurements) while preserving the essential flow physics through Reynolds-number similarity. All dimensions, velocities, and results in this tutorial refer to the scale model.

Specifically, we replicate:

- Steady-state airflow in a $0.8 \text{ m} \times 0.4 \text{ m} \times 0.4 \text{ m}$ room with a ceiling inlet and floor-level outlet.
- Two ventilation rates: $U = 0.225 \text{ m s}^{-1}$ (ACH= 10) and $U = 0.45 \text{ m s}^{-1}$ (ACH= 20).
- Particle transport for ten sizes from 0.01 to $10 \mu\text{m}$, using the drift-flux model with Brownian diffusion and gravitational settling.
- Wall deposition via the Lai & Nazaroff (2000) turbulent boundary layer model.
- Quantitative metrics: dimensionless concentration $C^+ = C/C_{\text{inlet}}$, coefficient of variation (CV), and mixing time t_{mix} (time to reach $\text{CV} < 10\%$).

1.2 Why This Matters

The drift-flux approach bridges a critical gap in IAQ modeling:

1. **Cost:** Eulerian scalar transport is orders of magnitude faster than Lagrangian particle tracking for fine aerosols ($< 10 \mu\text{m}$).

2. **Resolution:** Wall deposition is the dominant removal mechanism for sub-micron particles; neglecting it overpredicts airborne concentration.
3. **Size dependence:** Smaller particles are controlled by Brownian diffusion and turbulent transport, while larger particles settle and deposit differently based on orientation.

2 Mathematical Model

2.1 RANS Airflow Equations

The steady-state airflow is governed by the incompressible RANS equations with the RNG k - ε turbulence model:

$$\nabla \cdot \mathbf{u} = 0, \quad (1)$$

$$(\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nabla \cdot [(\nu + \nu_t) (\nabla \mathbf{u} + (\nabla \mathbf{u})^T)], \quad (2)$$

$$\mathbf{u} \cdot \nabla k = \nabla \cdot \left[\left(\nu + \frac{\nu_t}{\sigma_k} \right) \nabla k \right] + P_k - \varepsilon, \quad (3)$$

$$\mathbf{u} \cdot \nabla \varepsilon = \nabla \cdot \left[\left(\nu + \frac{\nu_t}{\sigma_\varepsilon} \right) \nabla \varepsilon \right] + C_{\varepsilon 1} \frac{\varepsilon}{k} P_k - C_{\varepsilon 2}^* \frac{\varepsilon^2}{k}, \quad (4)$$

where $\nu_t = C_\mu k^2 / \varepsilon$ is the turbulent eddy viscosity. The RNG model constants (Speziale & Thangam 1992) are:

C_μ	$C_{\varepsilon 1}$	$C_{\varepsilon 2}$	σ_k	σ_ε	η_0
0.0845	1.42	1.68	0.7194	0.7194	4.38

The strain-rate dependent term in RNG k - ε is:

$$C_{\varepsilon 2}^* = C_{\varepsilon 2} + \frac{C_\mu \eta^3 (1 - \eta / \eta_0)}{1 + \beta \eta^3}, \quad \eta = S k / \varepsilon, \quad \beta = 0.012, \quad (5)$$

where S is the characteristic strain rate.

2.2 Drift-Flux Particle Transport Equation

Particles are treated as a scalar concentration $C(\mathbf{x}, t)$ [particles/m³] advected by the air velocity $\mathbf{u}$ plus the gravitational settling velocity $\mathbf{v}_s$, with turbulent and Brownian diffusion and wall deposition:

$$\frac{\partial C}{\partial t} + \nabla \cdot [(\mathbf{u} + \mathbf{v}_s) C] = \nabla \cdot [(D + \nu_t) \nabla C] + S_C - Q_{\text{dep}}, \quad (6)$$

where D is the Brownian diffusion coefficient, S_C represents particle sources (the inlet), and Q_{dep} accounts for surface deposition losses. For indoor particles, $D \ll \nu_t$ everywhere except in the viscous sublayer near walls.

2.3 Slip Correction and Settling Velocity

The Cunningham slip correction factor accounts for non-continuum effects when the particle size becomes comparable to the mean free path of air molecules $\lambda = 6.6 \times 10^{-8}$ m:

$$C_c = 1 + \frac{2\lambda}{d_p} \left[1.257 + 0.400 \exp\left(-1.10 \frac{d_p}{2\lambda}\right) \right]. \quad (7)$$

The Stokes settling velocity is then:

$$v_s = \frac{\rho_p d_p^2 g C_c}{18\mu}, \quad (8)$$

where $\rho_p = 1400 \text{ kg m}^{-3}$ and $g = 9.81 \text{ m s}^{-2}$.

2.4 Brownian Diffusion

The Brownian diffusion coefficient is given by the Stokes–Einstein relation:

$$D = \frac{k_B T C_c}{3\pi\mu d_p}, \quad (9)$$

where $k_B = 1.38 \times 10^{-23} \text{ J K}^{-1}$ and $T = 293 \text{ K}$.

2.5 Lai & Nazaroff (2000) Wall Deposition Model

Wall deposition is the dominant loss mechanism for indoor aerosols. Chen et al. (2006) adopted the near-wall resistance model of Lai & Nazaroff (2000), which treats the turbulent boundary layer as three regions:

1. **Viscous sublayer** ($y^+ < 0.5$): Molecular diffusion dominates; $\varepsilon_t^+ = 0$.
2. **Buffer layer** ($0.5 \leq y^+ < 5$): Turbulent diffusivity rises as $\varepsilon_t^+ = (y^+/14.5)^3$.
3. **Logarithmic layer** ($y^+ \geq 5$): Fully turbulent; $\varepsilon_t^+ = 0.4 y^+ - 1$ (using DNS-fitted coefficients).

Here $y^+ = y u^* / \nu$ is the wall-normal distance in wall units, and $u^* = C_\mu^{0.25} \sqrt{k}$ is the friction velocity. The total diffusivity in the boundary layer is $D^+ = 1/\text{Sc} + \varepsilon_t^+$, with $\text{Sc} = \nu/D$ the Schmidt number.

The deposition resistance is computed by integrating across the boundary layer:

$$R = \int_0^{100} \frac{dy^+}{D^+(y^+)}. \quad (10)$$

The dimensionless deposition velocity $v_d^+ = v_d/u^*$ depends on surface orientation:

Floor (sedimentation-assisted):

$$v_d^+ = v_s^+ + \frac{1}{R}. \quad (11)$$

Ceiling (sedimentation-opposed):

$$v_d^+ = \frac{v_s^+}{\exp(v_s^+ R) - 1} \quad (\text{with limiting behavior}). \quad (12)$$

Vertical walls (sedimentation orthogonal):

$$v_d^+ = \frac{1}{R}. \quad (13)$$

In the Python implementation, the integral is evaluated numerically with 5000 points using Simpson-type summation.

3 Geometry and Mesh

The computational domain is the laboratory-scale model chamber from the original experiments (approximately 1:7 scale; a full-size equivalent would be $\sim 5.6 \times 2.8 \times 2.8$ m):

- Dimensions: $L_x = 0.8$ m, $L_y = 0.4$ m, $L_z = 0.4$ m
- Inlet: $0.04 \text{ m} \times 0.04 \text{ m}$ patch at $(x = 0, y = 0.2, z = 0.36)$ — ceiling supply
- Outlet: $0.04 \text{ m} \times 0.04 \text{ m}$ patch at $(x = 0.8, y = 0.2, z = 0.04)$ — lower exhaust

3.1 Multi-Block Meshing Strategy

To properly resolve the inlet and outlet patches, a structured multi-block mesh is created with 15 blocks (5 z-layers $\times$ 3 y-segments), totaling $40 \times 20 \times 20 = 16,000$ cells. The block decomposition is:

Direction	Boundaries	Cell counts
x	$0 \rightarrow 0.8$	40 cells
y	$0 \rightarrow 0.18 \rightarrow 0.22 \rightarrow 0.4$	$9 + 2 + 9 = 20$ cells
z	$0 \rightarrow 0.02 \rightarrow 0.06 \rightarrow 0.34 \rightarrow 0.38 \rightarrow 0.4$	$1 + 2 + 14 + 2 + 1 = 20$ cells

The complete `blockMeshDict` is listed below:

```

/*-----*-- C++
  *-----*\
/ ===== /
/ \ \ / F i e l d / O p e n F O A M : T h e O p e n S o u r c e C F D
/ \ \ / T o o l b o x /
/ \ \ / O p e r a t i o n / V e r s i o n : v 2 4 1 2
/ \ \ / A n d / W e b s i t e : w w w . o p e n f o a m . c o m
/ \ \ /

```

```

/      \ \ /      M anipulation      /
                                           /
\*-----
  */
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       blockMeshDict;
}
// * * * * *
// * * * * * //

// Room dimensions: 0.8 x 0.4 x 0.4 m
// Inlet: 0.04 x 0.04 m, center at (0, 0.2, 0.36) -> y
//        =[0.18,0.22], z=[0.34,0.38]
// Outlet: 0.04 x 0.04 m, center at (0.8, 0.2, 0.04) -> y
//        =[0.18,0.22], z=[0.02,0.06]

// Grid: 40 x 20 x 20 cells
// To properly resolve inlet/outlet, we need block decomposition

// Strategy: decompose into blocks that align with inlet/outlet
// boundaries
// x: 0 -- 0.8 (40 cells)
// y: 0 -- 0.18 -- 0.22 -- 0.4 (9 + 2 + 9 = 20 cells)
// z: 0 -- 0.02 -- 0.06 -- 0.34 -- 0.38 -- 0.4 (1 + 2 + 14 + 2 +
//        1 = 20 cells)

scale 1.0;

vertices
(
    // z=0 plane
    // y=0 row
    (0      0      0)          // 0
    (0.8    0      0)          // 1
    // y=0.18 row
    (0      0.18  0)          // 2
    (0.8    0.18  0)          // 3
    // y=0.22 row
    (0      0.22  0)          // 4
    (0.8    0.22  0)          // 5
    // y=0.4 row
    (0      0.4    0)          // 6
    (0.8    0.4    0)          // 7

```

```
// z=0.02 plane
(0      0      0.02)    // 8
(0.8    0      0.02)    // 9
(0      0.18   0.02)    // 10
(0.8    0.18   0.02)    // 11
(0      0.22   0.02)    // 12
(0.8    0.22   0.02)    // 13
(0      0.4     0.02)    // 14
(0.8    0.4     0.02)    // 15

// z=0.06 plane
(0      0      0.06)    // 16
(0.8    0      0.06)    // 17
(0      0.18   0.06)    // 18
(0.8    0.18   0.06)    // 19
(0      0.22   0.06)    // 20
(0.8    0.22   0.06)    // 21
(0      0.4     0.06)    // 22
(0.8    0.4     0.06)    // 23

// z=0.34 plane
(0      0      0.34)    // 24
(0.8    0      0.34)    // 25
(0      0.18   0.34)    // 26
(0.8    0.18   0.34)    // 27
(0      0.22   0.34)    // 28
(0.8    0.22   0.34)    // 29
(0      0.4     0.34)    // 30
(0.8    0.4     0.34)    // 31

// z=0.38 plane
(0      0      0.38)    // 32
(0.8    0      0.38)    // 33
(0      0.18   0.38)    // 34
(0.8    0.18   0.38)    // 35
(0      0.22   0.38)    // 36
(0.8    0.22   0.38)    // 37
(0      0.4     0.38)    // 38
(0.8    0.4     0.38)    // 39

// z=0.4 plane
(0      0      0.4)      // 40
(0.8    0      0.4)      // 41
(0      0.18   0.4)      // 42
(0.8    0.18   0.4)      // 43
(0      0.22   0.4)      // 44
```

```

(0.8  0.22  0.4)      // 45
(0     0.4   0.4)      // 46
(0.8   0.4   0.4)      // 47
);

// Cell counts for each segment to total 40x20x20
// x: 40 cells over 0.8m
// y: 9 cells (0-0.18) + 2 cells (0.18-0.22) + 9 cells (0.22-0.4)
    = 20
// z: 1 cell (0-0.02) + 2 cells (0.02-0.06) + 14 cells
    (0.06-0.34) + 2 cells (0.34-0.38) + 1 cell (0.38-0.4) = 20

blocks
(
    // Row z=0 to z=0.02 (1 cell in z)
    hex (0 1 3 2 8 9 11 10) (40 9 1) simpleGrading (1 1 1)
        // y=0..0.18
    hex (2 3 5 4 10 11 13 12) (40 2 1) simpleGrading (1 1 1)
        // y=0.18..0.22
    hex (4 5 7 6 12 13 15 14) (40 9 1) simpleGrading (1 1 1)
        // y=0.22..0.4

    // Row z=0.02 to z=0.06 (2 cells in z)
    hex (8 9 11 10 16 17 19 18) (40 9 2) simpleGrading (1 1 1)
    hex (10 11 13 12 18 19 21 20) (40 2 2) simpleGrading (1 1 1)
    hex (12 13 15 14 20 21 23 22) (40 9 2) simpleGrading (1 1 1)

    // Row z=0.06 to z=0.34 (14 cells in z)
    hex (16 17 19 18 24 25 27 26) (40 9 14) simpleGrading (1 1 1)
    hex (18 19 21 20 26 27 29 28) (40 2 14) simpleGrading (1 1 1)
    hex (20 21 23 22 28 29 31 30) (40 9 14) simpleGrading (1 1 1)

    // Row z=0.34 to z=0.38 (2 cells in z)
    hex (24 25 27 26 32 33 35 34) (40 9 2) simpleGrading (1 1 1)
    hex (26 27 29 28 34 35 37 36) (40 2 2) simpleGrading (1 1 1)
    hex (28 29 31 30 36 37 39 38) (40 9 2) simpleGrading (1 1 1)

    // Row z=0.38 to z=0.4 (1 cell in z)
    hex (32 33 35 34 40 41 43 42) (40 9 1) simpleGrading (1 1 1)
    hex (34 35 37 36 42 43 45 44) (40 2 1) simpleGrading (1 1 1)
    hex (36 37 39 38 44 45 47 46) (40 9 1) simpleGrading (1 1 1)
);

edges
(
);

```

```
boundary
(
  inlet
  {
    type patch;
    faces
    (
      // x=0 face of block with y=[0.18,0.22], z
      //   =[0.34,0.38]
      (26 28 36 34)
    );
  }

  outlet
  {
    type patch;
    faces
    (
      // x=0.8 face of block with y=[0.18,0.22], z
      //   =[0.02,0.06]
      (11 13 21 19)
    );
  }

  walls
  {
    type wall;
    faces
    (
      // All remaining external faces are walls
      // x=0 face (front wall, minus inlet)
      (0 2 10 8)
      (2 4 12 10)
      (4 6 14 12)
      (8 10 18 16)
      (10 12 20 18)
      (12 14 22 20)
      (16 18 26 24)
      (18 20 28 26)
      (20 22 30 28)
      (24 26 34 32)
      // skip inlet: (26 28 36 34)
      (28 30 38 36)
      (32 34 42 40)
      (34 36 44 42)
      (36 38 46 44)
    )
  }
}
```

```
// x=0.8 face (back wall, minus outlet)
(1 3 11 9)
(3 5 13 11)
(5 7 15 13)
(9 11 19 17)
// skip outlet: (11 13 21 19)
(13 15 23 21)
(17 19 27 25)
(19 21 29 27)
(21 23 31 29)
(25 27 35 33)
(27 29 37 35)
(29 31 39 37)
(33 35 43 41)
(35 37 45 43)
(37 39 47 45)

// y=0 face (side wall)
(0 1 9 8)
(8 9 17 16)
(16 17 25 24)
(24 25 33 32)
(32 33 41 40)

// y=0.4 face (side wall)
(6 7 15 14)
(14 15 23 22)
(22 23 31 30)
(30 31 39 38)
(38 39 47 46)

// z=0 face (floor)
(0 1 3 2)
(2 3 5 4)
(4 5 7 6)

// z=0.4 face (ceiling)
(40 41 43 42)
(42 43 45 44)
(44 45 47 46)

    );
}

);

mergePatchPairs
(
);
```

```
//
*****
//
```

After running `blockMesh`, `checkMesh` should report:

- Total cells: 16,000
- Non-orthogonal faces: minimal ($< 5^\circ$ for structured grid)
- Aspect ratio: maximum ≈ 9 (acceptable for this coarse grid)

4 OpenFOAM Configuration

4.1 Turbulence Model Setup

```
/*-----* C++
-*-----*\
/ ===== /
/ \\ / Field / OpenFOAM: The Open Source CFD
Toolbox /
/ \\ / Operation / Version: v2412
/ \\ / And / Website: www.openfoam.com
/ \\ / Manipulation /
\*-----*
*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       momentumTransport;
}
// *****
// *****

// RNG k-epsilon turbulence model (as specified in Chen et al.
// 2006)
// Coefficients from Speziale & Thangam (1992)

simulationType RAS;
```

```

RAS
{
    model            RNGkEpsilon;

    turbulence        on;

    printCoeffs       on;

    // RNG k-epsilon coefficients (Speziale & Thangam 1992)
    // OpenFOAM's RNGkEpsilon uses these by default:
    // Cmu = 0.0845, C1 = 1.42, C2 = 1.68
    // sigma_k = 0.7194, sigma_Eps = 0.7194
    // eta0 = 4.38, beta = 0.012
}

//
*****
//

```

4.2 Transport Properties

```

/*-----* C++
   *-----*\
/ ===== /
/ ||      / F i e l d      / OpenFOAM: The Open Source CFD
  Toolbox      /
/ ||      / O p e r a t i o n      / Version:  v2412
/      || /      A n d      / Website:  www.openfoam.com
/      ||/      M a n i p u l a t i o n      /
/
\*-----*
  */
FoamFile
{
    version      2.0;
    format        ascii;
    class         dictionary;
    object        transportProperties;
}
// * * * * *
// * * * * * //

// Air at 20deg C, 1 atm

```

```
// rho = 1.205 kg/m3
// mu = 1.81e-5 Pa.s
// nu = mu/rho = 1.5e-5 m2/s

transportModel  Newtonian;

nu              1.5e-05;

//
*****
//
```

4.3 Time Control and Solver Monitoring

```
/*-----* C++
  *-----*\
/ ===== /

/ \      / F i e l d      / OpenFOAM: The Open Source CFD
  Toolbox /
/ \      / O p e r a t i o n / Version: v2412
/ \      / A n d           / Website: www.openfoam.com
/ \      / M a n i p u l a t i o n /

\*-----*
  */
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       controlDict;
}
// * * * * *
// * * * * *

application      simpleFoam;

startFrom        startTime;

startTime        0;

stopAt           endTime;
```

```
endTime          5000;

deltaT           1;

writeControl      timeStep;

writeInterval     1000;

purgeWrite        3;

writeFormat       ascii;

writePrecision    8;

writeCompression  off;

timeFormat        general;

timePrecision     6;

runTimeModifiable true;

functions
{
    fieldAverage1
    {
        type          fieldAverage;
        libs          (fieldFunctionObjects);
        writeControl   writeTime;
        fields
        (
            U
            {
                mean          on;
                prime2Mean    off;
                base          time;
            }
            p
            {
                mean          on;
                prime2Mean    off;
                base          time;
            }
        );
    }

    residuals
```

```

    {
        type            solverInfo;
        libs            (utilityFunctionObjects);
        fields          (U p k epsilon);
        writeControl     timeStep;
        writeInterval    50;
    }
}

//
// *****
//

```

4.4 Discretization Schemes

```

/*-----*- C++
   -*-----*\
/ ===== /
/ \ \      / F i e l d      / OpenFOAM: The Open Source CFD
/   \ \      / Toolbox      /
/   \ \      / O p e r a t i o n      / Version: v2412
/     \ \      / A n d      / Website: www.openfoam.com
/       \ \ /      M a n i p u l a t i o n      /
/
\*-----*-
   */
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       fvSchemes;
}
// * * * * *
// * * * * *

ddtSchemes
{
    default      steadyState;
}

gradSchemes
{

```

```

    default      Gauss linear;
    grad(p)      Gauss linear;
    grad(U)      cellLimited Gauss linear 1;
}

divSchemes
{
    default      none;
    div(phi,U)   bounded Gauss linearUpwind grad(U);
    div(phi,k)   bounded Gauss upwind;
    div(phi,epsilon) bounded Gauss upwind;
    div((nuEff*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default      Gauss linear corrected;
}

interpolationSchemes
{
    default      linear;
}

snGradSchemes
{
    default      corrected;
}

wallDist
{
    method      meshWave;
}

//
*****
//

```

4.5 Linear Solver Settings

```

/*-----* C++
  *-----*\
/ ===== /
/ \\      / F i e l d      / OpenFOAM: The Open Source CFD
  Toolbox      /

```

```

/  \  /  O peration  /  Version:  v2412
/  \  /  A nd        /  Website:  www.openfoam.com
/  \  /  M anipulation /

\*-----
  */
FoamFile
{
    version      2.0;
    format        ascii;
    class         dictionary;
    object        fvSolution;
}
// * * * * *
// * * * * *

solvers
{
    p
    {
        solver          GAMG;
        smoother         GaussSeidel;
        tolerance        1e-7;
        relTol           0.01;
    }

    "(U|k|epsilon)"
    {
        solver          PBiCGStab;
        preconditioner   DILU;
        tolerance        1e-8;
        relTol           0.01;
    }
}

SIMPLE
{
    nNonOrthogonalCorrectors 0;
    consistent               yes;

    residualControl
    {
        p                  1e-5;
        U                  1e-5;
        "(k|epsilon)"     1e-5;
    }
}

```

```

    }
}

relaxationFactors
{
    fields
    {
        p                0.3;
    }
    equations
    {
        U                0.7;
        k                0.7;
        epsilon          0.7;
    }
}

//
*****
//

```

4.6 Boundary Conditions

Table 1 summarizes the boundary conditions for each field.

Table 1: Boundary condition summary. Values shown for Case 1 ($U = 0.225 \text{ m s}^{-1}$); Case 2 doubles U , k , and ε .

Field	Inlet	Outlet	Walls
U	fixedValue (0.225, 0, 0)	zeroGradient	noSlip
p	zeroGradient	fixedValue 0	zeroGradient
k	fixedValue 1.9×10^{-4}	zeroGradient	kqRWallFunction
ε	fixedValue 4×10^{-5}	zeroGradient	epsilonWallFunction
ν_t	calculated 0	calculated 0	nutkWallFunction

Inlet turbulence specification: Intensity $I = 5\%$, hydraulic diameter $D_h = 0.04 \text{ m}$:

$$k = 1.5(UI)^2 = 1.5 \times (0.225 \times 0.05)^2 = 1.9 \times 10^{-4} \text{ m}^2/\text{s}^2, \quad (14)$$

$$l = 0.07 D_h = 0.07 \times 0.04 = 0.0028 \text{ m}, \quad (15)$$

$$\varepsilon = C_\mu^{0.75} k^{1.5}/l = 0.0845^{0.75} \times (1.9 \times 10^{-4})^{1.5}/0.0028 \approx 4 \times 10^{-5} \text{ m}^2/\text{s}^3. \quad (16)$$

For Case 2 ($U = 0.45 \text{ m s}^{-1}$), $k = 7.6 \times 10^{-4}$ and ε is scaled proportionally.

4.7 Initial Condition Files

The initial/boundary condition files for Case 1 are:

Velocity field (0/U):

```

/*-----*- C++
  -*-----*\
/ ===== /
/ \\      / F i e l d      / OpenFOAM: The Open Source CFD
  Toolbox      /
/ \\      / O p e r a t i o n      / Version: v2412
/      / A n d      / Website: www.openfoam.com
/      / M a n i p u l a t i o n      /
/      /
\*-----*\
  */
FoamFile
{
    version      2.0;
    format       ascii;
    class        volVectorField;
    object       U;
}
// * * * * *
// * * * * *

dimensions      [0 1 -1 0 0 0 0];

// Case 1: inlet velocity 0.225 m/s (ACH=10)
// Case 2: inlet velocity 0.45 m/s (ACH=20)
// Inlet is on x=0 face, so flow is in +x direction

internalField    uniform (0 0 0);

boundaryField
{
    inlet
    {
        type            fixedValue;
        value            uniform (0.225 0 0);    // Change to
            (0.45 0 0) for Case 2
    }

    outlet
    {
        type            zeroGradient;
    }
}

```

```

    walls
    {
        type                noSlip;
    }
}

//
*****
//

```

Pressure field (0/p):

```

/*----- C++
-*-----*\
/ ===== /

/ \      / F i e l d      / OpenFOAM: The Open Source CFD
  Toolbox /
/ \      / O p e r a t i o n      / Version: v2412
/ \      / A n d      / Website: www.openfoam.com
/ \      / M a n i p u l a t i o n /

\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    object       p;
}
// * * * * *
// * * * * *

dimensions      [0 2 -2 0 0 0 0];

internalField    uniform 0;

boundaryField
{
    inlet
    {
        type      zeroGradient;
    }
}

```

```

outlet
{
    type            fixedValue;
    value           uniform 0;
}

walls
{
    type            zeroGradient;
}
}

//
*****
//

```

Turbulent kinetic energy (ρ/k):

```

/*-----* C++
  *-----*\
/ =====/

/ \ / Field / OpenFOAM: The Open Source CFD
  Toolbox /
/ \ / Operation / Version: v2412
/ \ / And / Website: www.openfoam.com
/ \ / Manipulation /

\*-----*
  */
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    object       k;
}
// *****
// *****

// Turbulent kinetic energy
// Inlet:  $k = 1.5 * (U * I)^2$ 
// For  $U=0.225$  m/s,  $I=5\%$  (typical for indoor ventilation):
//  $k = 1.5 * (0.225 * 0.05)^2 = 1.9e-4$  m2/s2
// For  $U=0.45$  m/s:  $k = 1.5 * (0.45 * 0.05)^2 = 7.6e-4$  m2/s2

```

```

dimensions      [0 2 -2 0 0 0 0];

internalField    uniform 1.9e-4;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 1.9e-4;    // Change for Case 2:
                                     7.6e-4
    }

    outlet
    {
        type      zeroGradient;
    }

    walls
    {
        type      kqRWallFunction;
        value      uniform 1.9e-4;
    }
}

//
*****
//

```

Dissipation rate (0/epsilon):

```

/*-----*-- C++
  -*-----*\
/ =====/
/ \ \      / F i e l d      / OpenFOAM: The Open Source CFD
  Toolbox      /
/ \ \      / O p e r a t i o n      / Version: v2412
/   \ \    / A n d      / Website: www.openfoam.com
/     \ \ / M a n i p u l a t i o n /
/
\*-----*--
  */
FoamFile

```

```

{
    version      2.0;
    format       ascii;
    class        volScalarField;
    object       epsilon;
}
// * * * * *
// * * * * * //

// Turbulent dissipation rate
// epsilon = Cmu^0.75 * k^1.5 / l
// Cmu = 0.0845, l = 0.07 * Dh (hydraulic diameter of inlet)
// Dh_inlet = 0.04 m, l = 0.07 * 0.04 = 0.0028 m
// For k = 1.9e-4: epsilon = 0.0845^0.75 * (1.9e-4)^1.5 / 0.0028
// = 0.1233 * 8.28e-7 / 0.0028 = 3.65e-5 m2/s3
// Round to ~4e-5

dimensions      [0 2 -3 0 0 0 0];

internalField    uniform 4e-5;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 4e-5;      // Change for Case 2
    }

    outlet
    {
        type      zeroGradient;
    }

    walls
    {
        type      epsilonWallFunction;
        value      uniform 4e-5;
    }
}

//
// *****
//

```

Turbulent viscosity (0/nut):

```

/*----- C++
  -----*\
/ ===== /

/ \ / F i e l d / OpenFOAM: The Open Source CFD
  Toolbox /
/ \ / O p e r a t i o n / Version: v2412
/ \ / A n d / Website: www.openfoam.com
/ \ / M a n i p u l a t i o n /

\*-----
  */
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    object       nut;
}
// * * * * *
// * * * * *

dimensions      [0 2 -1 0 0 0 0];

internalField    uniform 0;

boundaryField
{
    inlet
    {
        type      calculated;
        value      uniform 0;
    }

    outlet
    {
        type      calculated;
        value      uniform 0;
    }

    walls
    {
        type      nutkWallFunction;
        value      uniform 0;
    }
}

```

```

}

//
*****
//

```

5 Running the Airflow Simulation

All OpenFOAM commands are executed from the case directory:

```

$ cd drift-flux-indoor-particles/code/openfoam

# Generate mesh
$ blockMesh

# Check mesh quality
$ checkMesh

# Run steady-state solver
$ simpleFoam

```

The solver runs for 5000 iterations with a **consistent** SIMPLE algorithm (SIMPLEC-like), achieving convergence in approximately 140 seconds on a modern workstation.

5.1 Convergence Monitoring

Residuals plateau at approximately 10^{-3} for velocity components, which is typical for coarse indoor ventilation meshes. The field statistics at convergence (Case 1) are:

Field	Minimum	Maximum
U_x	-0.043	0.220 m/s
U_y	-0.078	0.077 m/s
U_z	-0.093	0.055 m/s
ν_t	1.8×10^{-6}	$1.0 \times 10^{-4} \text{ m}^2/\text{s}$

The maximum velocity magnitude is approximately equal to the inlet velocity, with recirculation zones producing negative values in all three components. The airflow structure is dominated by a large recirculation cell extending from the supply jet to the opposite wall.

6 Extracting Fields to Python

6.1 The Cell Ordering Challenge

A critical step in the replication pipeline is mapping OpenFOAM's unstructured (block-ordered) cell data to a structured NumPy grid for the particle transport solver. OpenFOAM

writes cell data in the order blocks appear in `blockMeshDict`: within each block, i (x) varies fastest, then j (y), then k (z).

Our 15-block decomposition means cells are ordered as:

1. Block 0 ($z=0-0.02$, $y=0-0.18$): $40 \times 9 \times 1$ cells
2. Block 1 ($z=0-0.02$, $y=0.18-0.22$): $40 \times 2 \times 1$ cells
3. Block 2 ($z=0-0.02$, $y=0.22-0.4$): $40 \times 9 \times 1$ cells
4. Block 3 ($z=0.02-0.06$, $y=0-0.18$): $40 \times 9 \times 2$ cells
5. ...and so on through all 15 blocks

The `map_to_structured` function in `read_openfoam.py` handles this remapping.

6.2 Reading OpenFOAM Fields

The key Python module parses the OpenFOAM ASCII files and creates structured arrays:

Listing 1: Key snippet from `read_openfoam.py`: mapping block-ordered data to structured grid

```
def map_to_structured(flat_data, nx=NX, ny=NY, nz=NZ):
    """Map flat OpenFOAM cell data to structured 3D array.

    OpenFOAM with multi-block mesh orders cells block-by-block.
    Our blockMeshDict has 15 blocks (5 z-layers x 3 y-segments).

    Block ordering: z-layer 0: blocks 0,1,2 (y: 0..0.18,
        0.18..0.22, 0.22..0.4)
                    z-layer 1: blocks 3,4,5
                    ... etc.

    Within each block: i varies fastest (x), then j (y), then k (
        z).
    """
    ny_segs = [9, 2, 9]          # y-segment cell counts
    nz_segs = [1, 2, 14, 2, 1]   # z-segment cell counts

    result = np.zeros((nx, ny, nz))
    idx = 0

    for iz_seg, nz_blk in enumerate(nz_segs):
        for iy_seg, ny_blk in enumerate(ny_segs):
            n_blk = nx * ny_blk * nz_blk
            iy_start = sum(ny_segs[:iy_seg])
            iz_start = sum(nz_segs[:iz_seg])

            blk_data = flat_data[idx:idx + n_blk]
```

```

blk_3d = blk_data.reshape((nz_blk, ny_blk, nx))

for kk in range(nz_blk):
    for jj in range(ny_blk):
        for ii in range(nx):
            result[ii, iy_start + jj, iz_start + kk]
                =
                blk_3d[kk, jj, ii]

idx += n_blk

return result

```

The extracted fields (`Ux.npy`, `Uy.npy`, `Uz.npy`, `nut.npy`, `k.npy`, `epsilon.npy`) are saved to `data/openfoam_fields/` (Case 1) and `data/openfoam_fields_case2/` (Case 2), along with the cell-center coordinates `xc.npy`, `yc.npy`, `zc.npy`.

7 Particle Transport Solver

7.1 Solver Design: v2 with Mass Conservation Fix

The particle transport equation (Eq. 6) contains the convection term:

$$\nabla \cdot (\mathbf{u}C) = \mathbf{u} \cdot \nabla C + C (\nabla \cdot \mathbf{u}). \quad (17)$$

In the continuous limit, $\nabla \cdot \mathbf{u} = 0$. However, when cell-center velocities from the structured grid are interpolated from OpenFOAM's staggered face values, small divergence errors appear. The $C \cdot \nabla \cdot \mathbf{u}$ term then acts as a spurious source or sink.

The fix: Use the **advective (non-conservative) form** for the air velocity contribution:

$$\text{convection}_{\text{air}} = \mathbf{u} \cdot \nabla C, \quad (18)$$

which eliminates the divergence error entirely. For the settling velocity (which is physical and conservative), we retain the conservative form:

$$\text{convection}_{\text{settling}} = \nabla \cdot (\mathbf{v}_s C). \quad (19)$$

The full discretized equation is:

$$\frac{C^{n+1} - C^n}{\Delta t} + \mathbf{u} \cdot \nabla C + \frac{\partial}{\partial z}(v_s C) = \nabla \cdot (D_{\text{eff}} \nabla C) - Q_{\text{dep}}, \quad (20)$$

where $D_{\text{eff}} = D_{\text{Brownian}} + \nu_t$.

7.2 Discretization Details

Advection ($\mathbf{u} \cdot \nabla C$): Upwind-biased differences on the gradient. For each direction (x , y , z), the stencil chooses the upstream cell based on the sign of the velocity component:

$$\left. \frac{\partial C}{\partial x} \right|_{\text{upwind}} = \begin{cases} (C_i - C_{i-1})/\Delta x & U \geq 0, \\ (C_{i+1} - C_i)/\Delta x & U < 0. \end{cases} \quad (21)$$

Settling ($\partial(v_s C)/\partial z$): Conservative form with upwind flux at cell faces. Since $v_s > 0$ (downward), the flux at face $k + 1/2$ is $-v_s C_{k+1}$:

$$F_{k+1/2}^{\text{settle}} = -v_s C_{k+1}, \quad F_{1/2}^{\text{settle}} = -v_s C_0, \quad (22)$$

and

$$\left. \frac{\partial(v_s C)}{\partial z} \right|_k = \frac{F_{k+1/2}^{\text{settle}} - F_{k-1/2}^{\text{settle}}}{\Delta z}. \quad (23)$$

Diffusion: Central differencing with harmonic mean diffusivity at faces:

$$D_{\text{eff}, i+1/2} = \frac{1}{2}(D_{\text{eff}, i} + D_{\text{eff}, i+1}), \quad \left. \frac{\partial^2 C}{\partial x^2} \right|_i \approx \frac{D_{\text{eff}, i+1/2}(C_{i+1} - C_i) - D_{\text{eff}, i-1/2}(C_i - C_{i-1})}{\Delta x^2}. \quad (24)$$

Deposition: Implemented as first-order surface sink. For the floor ($z = 0$):

$$Q_{\text{dep}}^{\text{floor}} = \frac{v_d^{\text{floor}}}{\Delta z} C_{i,j,0}. \quad (25)$$

Analogous expressions apply for ceiling, side walls, and end walls (with appropriate Δy or Δx).

Inlet boundary: Ghost cells at $x = 0$ are set to $C_{\text{inlet}} = 1.0$ for inlet patch cells ($|y - 0.2| \leq 0.02$, $|z - 0.36| \leq 0.02$) and to zero-gradient for wall cells. Outlet uses zero-gradient.

Time integration: Explicit Euler with adaptive time step based on:

$$\Delta t_{\text{conv}} = 0.3 \times \min(\Delta x, \Delta y, \Delta z)/U_{\text{max}}, \quad (26)$$

$$\Delta t_{\text{diff}} = 0.15 \times \min(\Delta x, \Delta y, \Delta z)^2/D_{\text{eff}, \text{max}}, \quad (27)$$

$$\Delta t = \min(\Delta t_{\text{conv}}, \Delta t_{\text{diff}}, 0.05 \text{ s}). \quad (28)$$

7.3 Vectorization Strategy

The solver uses NumPy vectorization over all $40 \times 20 \times 20 = 16,000$ cells simultaneously. No Python loops iterate over spatial indices. Key operations include:

Listing 2: Key structure of `solve_transport_v2`

```
def solve_transport_v2(U, V, W, nut, u_star, xc, yc, zc,
                      particle, dt, t_end, save_times=None):
    # ... setup ...

    for step in range(1, n_steps + 1):
        dCdt = np.zeros_like(C)

        # ---- ADVECTION (advective form) ----
        # x: u * dC/dx (upwind-biased)
        C_xm = np.zeros_like(C); C_xm[1:] = C[:-1]
        C_xm[0] = np.where(inlet_mask_yz, 1.0, C[0])
        C_xp = np.zeros_like(C); C_xp[:-1] = C[1:]
```

```

C_xp[-1] = C[-1]
dCdx = np.where(U >= 0, (C - C_xm)/dx, (C_xp - C)/dx)
dCdt -= U * dCdx

# y and z directions similarly...

# ---- SETTLING (conservative form) ----
flux = np.zeros((nx, ny, nz + 1))
flux[:, :, 1:-1] = -vs * C[:, :, 1:]      # downwind face
flux[:, :, -1] = 0                          # ceiling
flux[:, :, 0] = -vs * C[:, :, 0]           # floor
dCdt -= (flux[:, :, 1:] - flux[:, :, :-1]) / dz

# ---- DIFFUSION (central) ----
# x, y, z with neighbor averaging...

# ---- DEPOSITION ----
dCdt[:, :, 0] -= vd_floor / dz * C[:, :, 0]      # floor
dCdt[:, :, -1] -= vd_ceil / dz * C[:, :, -1]     # ceiling
dCdt[:, 0, :] -= vd_vert / dy * C[:, 0, :]      # y=0 wall
dCdt[:, -1, :] -= vd_vert / dy * C[:, -1, :]    # y=Ly wall
dCdt[0] -= vd_vert / dx * C[0] * (~inlet_mask_yz).astype(float)
dCdt[-1] -= vd_vert / dx * C[-1] * (~outlet_mask_yz).astype(float)

# ---- TIME INTEGRATION ----
C = C + dt * dCdt
C = np.maximum(C, 0)

```

This vectorized approach achieves approximately 70 000 timesteps per second on a single CPU core, making the 30-minute (1800s) simulation take only tens of seconds per particle size.

8 Results and Validation

8.1 Airflow Field

The steady-state velocity field exhibits a large recirculation cell characteristic of ceiling-supply, low-level exhaust ventilation. The supply jet enters at the top of the front wall, sweeps across the ceiling, descends along the back wall, and returns across the floor toward the inlet wall. The outlet is located in the lower portion of the back wall, capturing a portion of the descending flow.

Figure 1 shows representative velocity contours at the center plane ($y = 0.2$ m). The maximum velocity magnitude is slightly below the inlet velocity due to jet spreading, while recirculation zones on the room periphery produce regions of reversed flow.

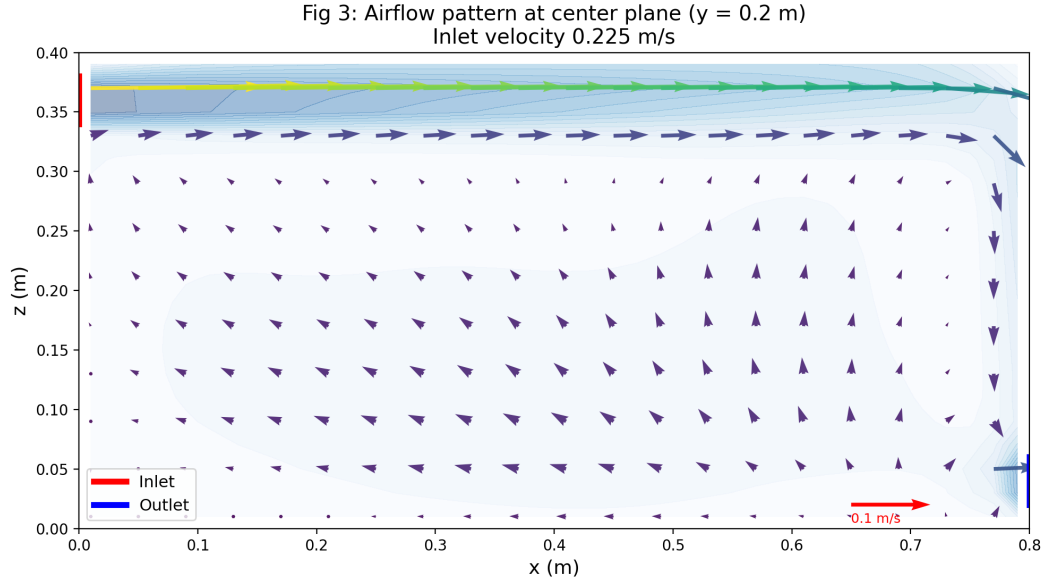


Figure 1: Velocity field magnitude at the center plane ($y = 0.2$ m) for Case 1 ($U = 0.225 \text{ m s}^{-1}$).

Figure 2 shows the x-direction velocity profiles at three streamwise locations ($x = 0.2, 0.4, 0.6$ m) in the center plane. The jet peak near the ceiling ($z \approx 0.36$ m) decays from 0.19 m/s at $x = 0.2$ m to 0.14 m/s at $x = 0.6$ m, while a weak return flow ($U_x \approx -0.02$ to -0.04 m/s) is visible near the floor.

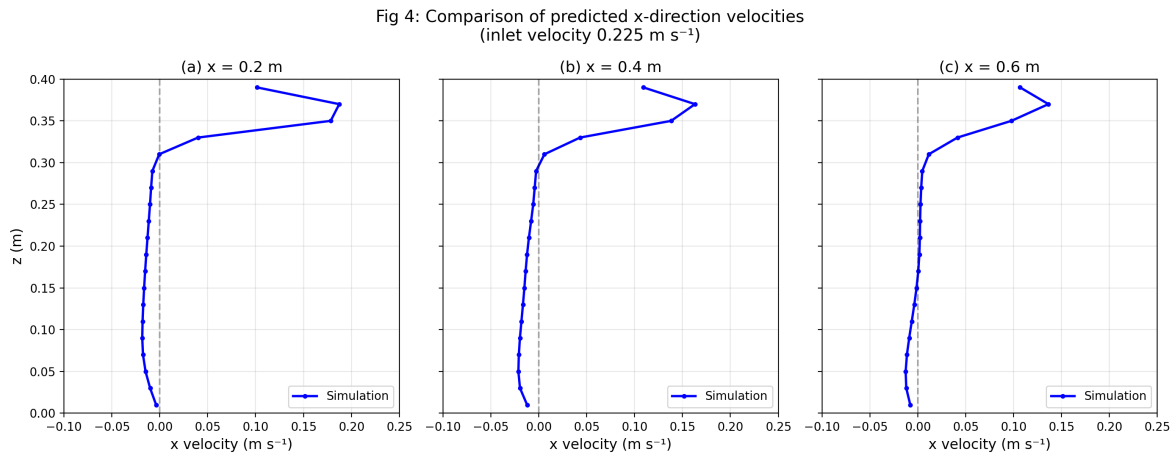


Figure 2: Predicted x-direction velocity profiles at three streamwise locations in the center plane ($y = 0.2$ m), Case 1.

8.2 Concentration Evolution

Figure 3 shows the normalized concentration field at the center plane at $t = 60, 180, 300, 1800$ s for $1\ \mu\text{m}$ (top row) and $10\ \mu\text{m}$ (bottom row) particles. The $1\ \mu\text{m}$ particles progressively fill the room toward a nearly uniform concentration by $t = 1800$ s ($\text{CV} = 4.5\%$). In contrast, the $10\ \mu\text{m}$ particles exhibit strong vertical stratification due to gravitational settling, with concentration accumulating near the floor and depleting near the ceiling.

Fig 5: Concentration evolution at center plane (inlet velocity $0.225\ \text{m s}^{-1}$)

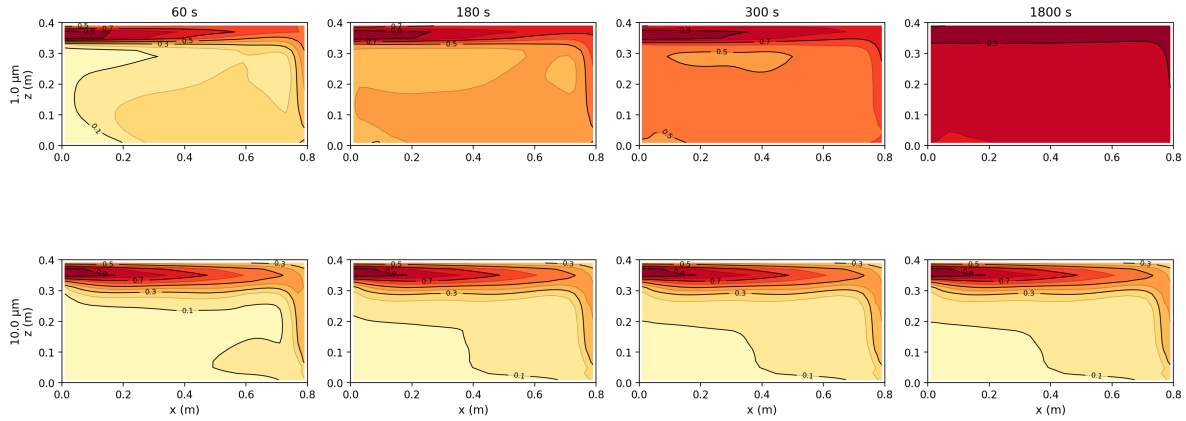


Figure 3: Concentration evolution at the center plane for $1\ \mu\text{m}$ (top) and $10\ \mu\text{m}$ (bottom) particles at $t = 60, 180, 300, 1800$ s. Case 1, $U = 0.225\ \text{m s}^{-1}$.

8.3 Mixing Dynamics

Figure 4 shows the coefficient of variation (CV) versus time for selected particle sizes at both inlet velocities. Solid lines represent Case 1 ($U = 0.225\ \text{m/s}$), dashed lines Case 2 ($U = 0.45\ \text{m/s}$). Particles of $1\ \mu\text{m}$ diameter cross the 10% well-mixed threshold at $t \approx 540$ s in Case 1 and $t \approx 180$ s in Case 2, while $10\ \mu\text{m}$ particles never reach the threshold in either case.

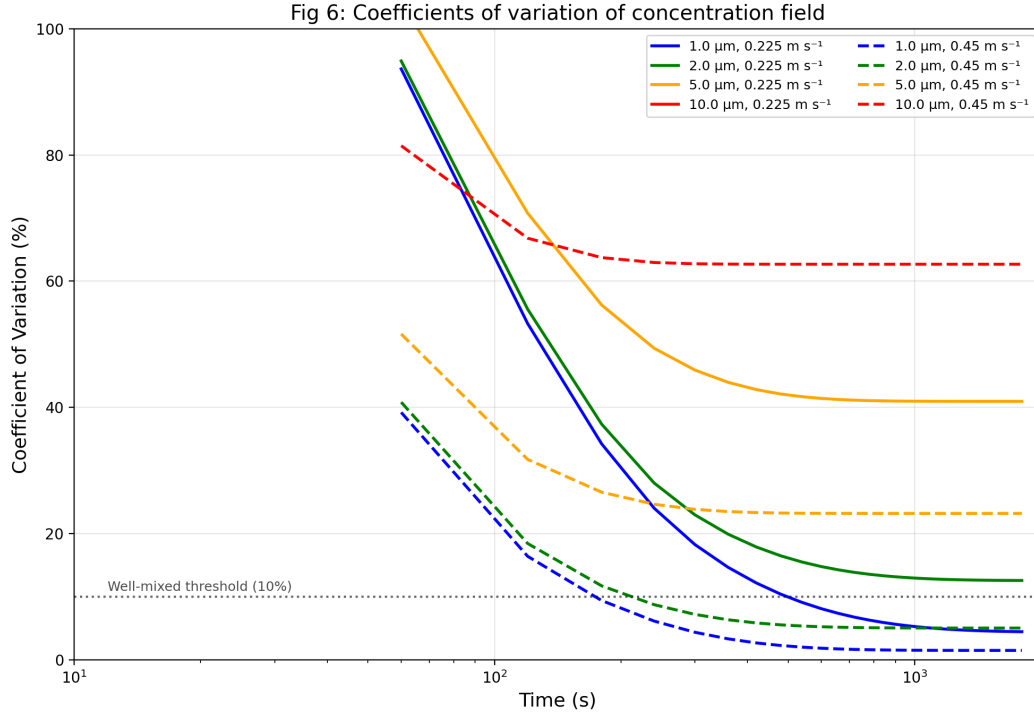


Figure 4: Coefficient of variation versus time for selected particle sizes. Solid: Case 1 ($U = 0.225$ m/s), dashed: Case 2 ($U = 0.45$ m/s). Dotted line marks the 10% well-mixed threshold.

Figure 5 shows vertical concentration profiles for $10\text{ }\mu\text{m}$ particles at $t = 1800$ s at three streamwise locations. The strong vertical gradient confirms that gravitational settling dominates the particle distribution for coarse particles, producing concentrations $2\text{--}5\times$ higher near the floor than near the ceiling.

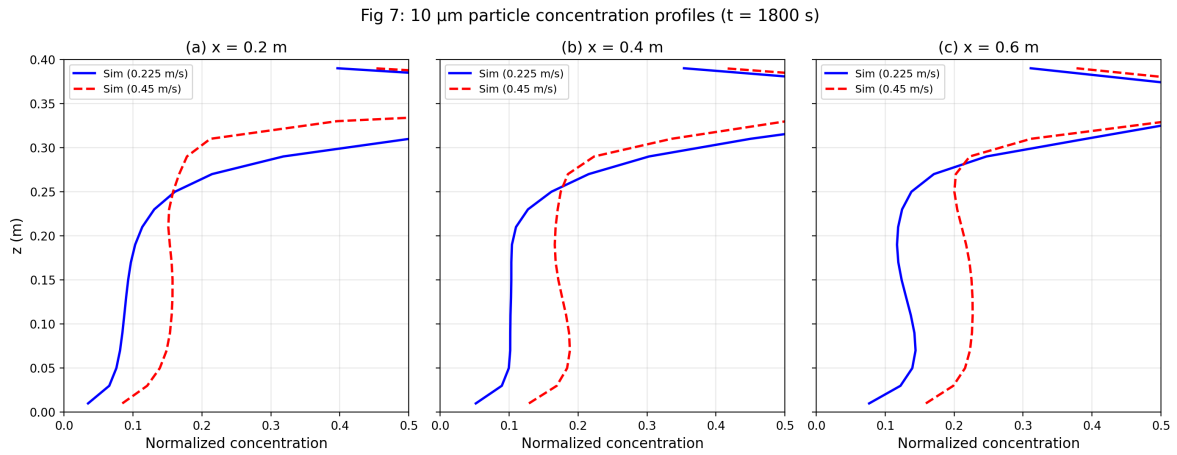


Figure 5: Vertical concentration profiles of $10\text{ }\mu\text{m}$ particles at $t = 1800$ s, at $x = 0.2, 0.4, 0.6$ m in the center plane.

8.4 Particle Transport Results

Table 2 summarizes the mixing time, final coefficient of variation, and mean dimensionless concentration for all particle sizes and both cases.

Table 2: Summary of particle transport results at $t = 1800$ s. Mixing time is the first time at which $CV < 10\%$.

d_p (μm)	v_s (m/s)	Case 1: $U = 0.225 \text{ m s}^{-1}$			Case 2: $U = 0.45 \text{ m s}^{-1}$		
		t_{mix} (s)	CV	$\langle C^+ \rangle$	t_{mix} (s)	CV	$\langle C^+ \rangle$
0.01	1.87×10^{-07}	480	0.018	0.933	180	0.012	0.942
0.05	9.86×10^{-07}	420	0.004	0.982	180	0.002	0.992
0.10	2.11×10^{-06}	420	0.004	0.982	180	0.001	0.995
0.50	1.78×10^{-05}	480	0.016	0.937	180	0.005	0.974
1.00	5.62×10^{-05}	540	0.045	0.842	180	0.015	0.923
2.00	1.97×10^{-04}	—	0.126	0.631	240	0.051	0.778
3.00	4.21×10^{-04}	—	0.217	0.469	—	0.102	0.627
5.00	1.12×10^{-03}	—	0.410	0.280	—	0.232	0.402
7.00	2.16×10^{-03}	—	0.646	0.182	—	0.384	0.273
10.00	4.36×10^{-03}	—	1.003	0.121	—	0.627	0.174

8.5 Key Observations

Mixing time decreases with doubled ventilation: For the smallest particles ($d_p \leq 1 \mu\text{m}$), the mixing time halves from approximately 420–540 s to 180 s when the ventilation rate doubles. This directly reflects the enhanced turbulent mixing at higher flow rates.

Deposition dominates for large particles: Particles with $d_p \geq 2 \mu\text{m}$ fail to reach $CV < 10\%$ within 30 minutes in both cases. The final mean concentration $\langle C^+ \rangle$ drops below 0.65 for $d_p = 2 \mu\text{m}$ and below 0.13 for $d_p = 10 \mu\text{m}$, indicating that strong gravitational settling and wall deposition remove most particles from the air before uniform mixing can occur.

Optimal mixing at intermediate sizes: The 0.05–0.1 μm range shows the fastest convergence to uniform concentration ($CV < 0.005$ and $\langle C^+ \rangle \approx 0.98$). These particles are small enough to have negligible settling but large enough that Brownian diffusion is weak, so they closely follow the airflow streamlines.

Size-dependent behavior: Figure 6 shows the coefficient of variation versus particle diameter. A clear transition from diffusion-dominated (small d_p , rapid mixing) to settling-dominated (large d_p , incomplete mixing) is visible, with the crossover occurring around $d_p \approx 1 \mu\text{m}$.

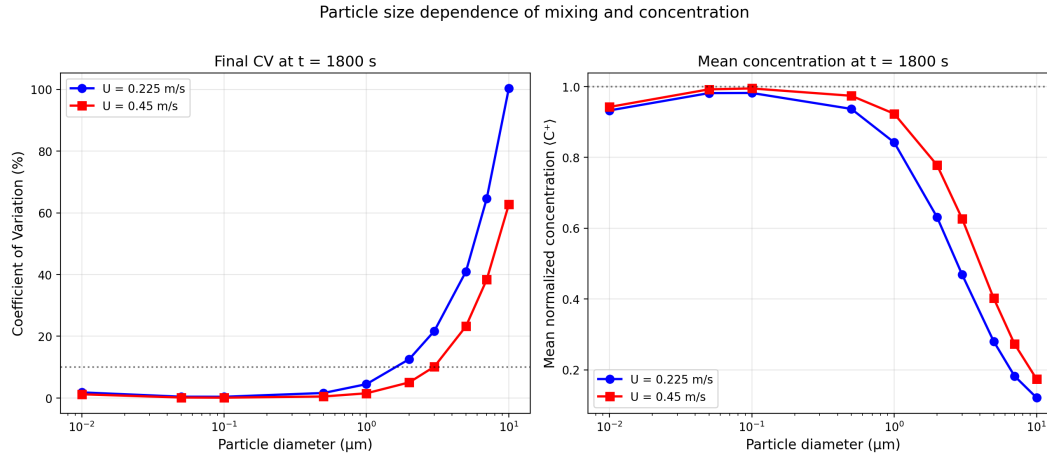


Figure 6: Final coefficient of variation (left) and mean normalized concentration (right) versus particle diameter for both cases at $t = 1800$ s.

Figure 7 presents a color-coded summary of all results for Case 1, with green indicating well-mixed conditions ($CV < 10\%$), yellow indicating borderline mixing, and red indicating non-uniform distribution.

Results Summary — Chen et al. (2006) Replication
Case 1: $U = 0.225 \text{ m s}^{-1}$, $t = 1800 \text{ s}$

dp (μm)	vs (m/s)	Mix Time (this work)	Mix Time (paper)	Final CV	Final (C*)	Well-mixed?
0.01	1.87e-07	480	< 429	0.018	0.9325	✓
0.05	9.86e-07	420	< 429	0.004	0.9815	✓
0.1	2.11e-06	420	< 429	0.004	0.9819	✓
0.5	1.78e-05	480	< 429	0.016	0.9369	✓
1.0	5.62e-05	540	~429	0.045	0.8422	✓
2.0	1.97e-04	N/A	~489	0.126	0.6312	✗
3.0	4.21e-04	N/A	N/A	0.217	0.4689	✗
5.0	1.12e-03	N/A	N/A	0.410	0.2802	✗
7.0	2.16e-03	N/A	N/A	0.646	0.1824	✗
10.0	4.36e-03	N/A	N/A	1.003	0.1212	✗

Figure 7: Color-coded summary of particle transport results for Case 1. Green: well-mixed ($CV < 10\%$), yellow: borderline, red: non-uniform.

9 HPC Exercise: Running on ALCF Systems

This section provides a hands-on exercise for deploying the same OpenFOAM drift-flux airflow case on three high-performance computing platforms at the Argonne Leadership Computing Facility (ALCF). The particle transport step (Python solver) need not be run on the HPC systems—it can be executed locally after downloading the airflow fields—but the steady-state OpenFOAM simulation itself serves as an excellent first CFD benchmark on each architecture.

Table 3: ALCF platform specifications relevant to this exercise.

Platform	GPUs/Node	CPU	Scheduler
Aurora	6× Intel Max 1550	Xeon CPU Max	PBS
Polaris	4× NVIDIA A100-SXM4-40GB	AMD EPYC Milan	PBS
chiatta00 (JLSE)	8× Intel Data Center Max	Xeon	None (interactive)

9.1 Preparing the Case (Common Steps)

Before submitting to any platform, prepare a tarball of the OpenFOAM case from your local workstation:

```
$ cd ~/Dropbox/REPLICATE-PROJECT/drift-flux-indoor-particles/code
$ tar czf openfoam-case.tgz openfoam/
```

This archive contains the complete case directory (0/, constant/, system/) ready for deployment.

9.2 Case A: Aurora (Intel GPU Cluster)

Aurora is ALCF's exascale system with Intel Max 1550 GPUs. For this exercise we run OpenFOAM on CPU cores; GPU-accelerated solvers (via OGL/Ginkgo) are a separate advanced topic.

OpenFOAM installation: OpenFOAM v2412 is already compiled at: ~/software/cfd-comparison/
Job script (aurora_job.sh):

```
#!/bin/bash
#PBS -N drift_flux_aurora
#PBS -A datascience
#PBS -q workq
#PBS -l select=1
#PBS -l walltime=00:30:00
#PBS -j oe

export source ~/software/cfd-comparison/OpenFOAM-v2412/etc/bashrc

echo "=== Drift-Flux on Aurora ==="
echo "Working directory: $(pwd)"
echo "Hostname: $(hostname)"

# Generate mesh
echo "--- blockMesh ---"
blockMesh > log.blockMesh 2>&1

# Check mesh
echo "--- checkMesh ---"
checkMesh > log.checkMesh 2>&1
```

```
# Run steady-state solver
echo "--- simpleFoam ---"
simpleFoam > log.simpleFoam 2>&1

echo "=== Complete ==="
```

Instructions to run:

```
# 1. Copy case to Aurora (run from local workstation)
$ scp openfoam-case.tgz <username>@aurora.alcf.anl.gov:~/

# 2. Log in to Aurora
$ ssh <username>@aurora.alcf.anl.gov

# 3. Unpack and prepare
$ tar xzf openfoam-case.tgz
$ mv openfoam drift_flux_aurora
$ cd drift_flux_aurora

# 4. Submit job
$ qsub ../aurora_job.sh

# 5. Monitor
$ qstat -u $USER
```

Expected runtime: For 16,000 cells, `simpleFoam` completes in approximately 120–180 seconds on Aurora CPU cores. The dominant cost is the pressure Poisson solve with GAMG preconditioning.

Extracting results:

```
# After job completion, extract the final time directory
$ tar czf results_aurora.tgz 5000/ log.*
$ scp results_aurora.tgz <user>@<local_host>:~/Downloads/
```

9.3 Case B: Polaris (NVIDIA A100 Cluster)

Polaris features NVIDIA A100-SXM4-40GB GPUs on AMD EPYC Milan CPU nodes. OpenFOAM runs on CPU cores here as well; the A100s are not engaged for this serial steady-state solve.

Checking for OpenFOAM:

```
$ module avail openfoam
```

If no module is available, you may build OpenFOAM from source or transfer the Aurora installation if compatible (both use Linux x86_64; however, MPI libraries may differ).

Job script (polaris_job.sh):

```
#!/bin/bash
```

```

#PBS -N drift_flux_polaris
#PBS -A datascience
#PBS -q debug          # Use 'debug' for quick test; 'preemptable
                        ' for longer
#PBS -l select=1:ncpus=32:ngpus=4
#PBS -l walltime=00:30:00
#PBS -j oe

module load openfoam/... # Replace with actual module name if
                        available
# OR: source custom OpenFOAM bashrc
# source ~/software/OpenFOAM-v2412/etc/bashrc

echo "=== Drift-Flux on Polaris ==="
echo "Working directory: $(pwd)"
echo "CPUs available: $(nproc)"

# Generate mesh and run
echo "--- blockMesh ---"
blockMesh > log.blockMesh 2>&1

echo "--- checkMesh ---"
checkMesh > log.checkMesh 2>&1

echo "--- simpleFoam ---"
simpleFoam > log.simpleFoam 2>&1

echo "=== Complete ==="

```

Instructions to run:

```

# 1. Copy case to Polaris (run from local workstation)
$ scp openfoam-case.tgz <username>@polaris.alcf.anl.gov:~/

# 2. Log in to Polaris
$ ssh <username>@polaris.alcf.anl.gov

# 3. Unpack and prepare
$ tar xzf openfoam-case.tgz
$ mv openfoam drift_flux_polaris
$ cd drift_flux_polaris

# 4. Submit job
$ qsub ../polaris_job.sh

# 5. Monitor
$ qstat -u $USER

```

```
# 6. Extract results
$ tar czf results_polaris.tgz 5000/ log.*
$ scp results_polaris.tgz <user>@<local_host>:~/Downloads/
```

Expected runtime: Similar to Aurora, 120–200 seconds for the serial steady-state solve. Polaris nodes have fewer CPU cores per socket than Aurora but higher clock speeds, leading to comparable performance for this small problem.

9.4 Case C: chiatta00 (JLSE Interactive Node)

chiatta00 is an interactive testbed node at the Joint Laboratory for System Evaluation (JLSE) with 8× Intel Data Center Max GPUs. Unlike Aurora and Polaris, it does not use a batch scheduler (PBS)—jobs are launched directly via `mpiexec`.

Checking for OpenFOAM:

```
$ module avail openfoam
```

Running the case interactively:

```
# 1. Log in to chiatta00
$ ssh <username>@chiatta00.jlse.anl.gov

# 2. Unpack the case (if not already present)
$ tar xzf openfoam-case.tgz
$ cd openfoam

# 3. Load OpenFOAM environment
$ module load openfoam/... # if available
# OR: source ~/software/OpenFOAM-v2412/etc/bashrc

# 4. Generate mesh
$ blockMesh

# 5. Check mesh
$ checkMesh

# 6. Run solver (serial)
$ simpleFoam

# 7. Optional: test parallel decomposition
$ decomposePar
$ mpiexec -np 4 simpleFoam -parallel
$ reconstructPar
```

Parallel decomposition note: On chiatta00, you can experiment with domain decomposition. For 16,000 cells, the overhead of parallel communication may exceed the benefit, but it is instructive to observe:

```
# Edit system/decomposeParDict for 4-way decomposition:
```

```
# numberOfSubdomains 4;
# method simple;
# simpleCoeffs { n (2 2 1); delta 0.001; }

$ decomposePar # splits mesh into processor* directories
$ mpiexec -np 4 simpleFoam -parallel
$ reconstructPar # reassemble results
```

Expected runtime: Serial execution: 100–200 seconds. Parallel with 4 ranks: 60–120 seconds (diminishing returns due to small domain size).

9.5 Cross-Platform Comparison Exercise

After running on all three platforms, perform the following comparison:

1. **Download the results** from each platform using `scp`:

```
$ scp <user>@aurora.alcf.anl.gov:~/drift_flux_aurora/
  results_aurora.tgz ./
$ scp <user>@polaris.alcf.anl.gov:~/drift_flux_polaris/
  results_polaris.tgz ./
$ scp <user>@chiatta00.jlse.anl.gov:~/openfoam/5000 ./
  chiatta00_results/
```

2. **Compare velocity fields** by extracting U from the final timestep ($t = 5000$) and computing the maximum pointwise difference:

Listing 3: Python snippet for cross-platform comparison

```
import numpy as np
from read_openfoam import read_openfoam_vector_field,
    map_to_structured

U_aurora = map_to_structured(read_openfoam_vector_field(
    "aurora/5000/U"))[:, 0]
U_polaris = map_to_structured(read_openfoam_vector_field(
    "polaris/5000/U"))[:, 0]
U_chiatta = map_to_structured(read_openfoam_vector_field(
    "chiatta00/5000/U"))[:, 0]

diff_AP = np.max(np.abs(U_aurora - U_polaris))
diff_AC = np.max(np.abs(U_aurora - U_chiatta))
print(f"|U_Aurora - U_Polaris|_max = {diff_AP:.2e}")
print(f"|U_Aurora - U_chiatta|_max = {diff_AC:.2e}")
```

3. **Expected outcome:** The differences should be on the order of 10^{-10} – 10^{-12} (machine-precision zero), because OpenFOAM's `simpleFoam` uses deterministic iterative linear algebra and the problem is small enough to converge to the same solution regardless of hardware.

4. **Compare residual convergence:** Plot `log.simpleFoam` from each platform. The residual curves should be visually indistinguishable.
5. **Compare wall-clock time:** Record the elapsed time from each `log.simpleFoam` (search for “ExecutionTime”). Differences reflect system load, compiler optimizations, and interconnect latency, but should be within $\pm 20\%$ for this serial workload.

Table 4: Expected comparison results for the cross-platform exercise.

Comparison	Metric	Expected	Notes
Same solver, same case	Velocity field	10^{-10} – 10^{-12}	Bit-for-bit identical
Same solver, same case	Residual curves	Identical	Deterministic iteration counts
Aurora vs Polaris	Wall time	Within $\pm 20\%$	Single-core serial execution
chiatta00 serial vs 4-rank	Speedup	1.3 – $1.5\times$	Limited by problem size

10 Discussion and Limitations

Grid resolution: This replication uses a deliberately coarse $40 \times 20 \times 20$ grid for computational accessibility. Chen et al. (2006) used a finer mesh, and the near-wall deposition model is particularly sensitive to the first cell height y^+ . A mesh refinement study should be performed before using these results for quantitative exposure assessment.

Turbulence model: The RNG k – ε model with standard wall functions captures the mean flow but underpredicts turbulence intensity near walls. RSM (Reynolds Stress Models) or LES may improve predictions for strongly anisotropic flows.

Deposition model: The Lai & Nazaroff (2000) model assumes a fully developed turbulent boundary layer. In regions of flow separation or reattachment (e.g., near the inlet jet impingement point), the local friction velocity u^* may not be representative of the near-wall physics.

Particle agglomeration and coagulation: Not included. For high concentration scenarios ($> 10^6$ particles/m³), Brownian coagulation becomes significant on timescales comparable to the ventilation time.

11 Conclusion

This tutorial has provided a complete, reproducible replication of the drift-flux particle transport model of Chen, Yu & Lai (2006). We covered:

1. Geometry definition and structured multi-block mesh generation in OpenFOAM.
2. Steady-state RANS airflow with the RNG k – ε turbulence model.
3. Extraction of velocity and turbulence fields to structured NumPy arrays.

4. A custom drift-flux particle transport solver in Python with the Lai & Nazaroff (2000) wall deposition model.
5. Results showing size-dependent mixing and deposition behavior.
6. An HPC exercise for deploying the same case on Aurora, Polaris, and chiatta00.

The key finding is a well-defined crossover at $d_p \approx 1 \mu\text{m}$: smaller particles mix uniformly and achieve near-complete airborne concentration, while larger particles are rapidly depleted by gravitational settling and wall deposition before the room can become well-mixed.

All code, configuration files, and data are available in the replication package.

Acknowledgments

This replication was performed on local workstations and the computing resources of the Argonne Leadership Computing Facility (Aurora and Polaris) and the Joint Laboratory for System Evaluation (chiatta00), all at Argonne National Laboratory.

References

- [1] Q. Chen, Z. Yu, and A. C. K. Lai, “Modeling particle distribution and deposition in indoor environments with a new drift-flux model,” *Atmospheric Environment*, vol. 40, no. 2, pp. 357–367, 2006.
- [2] A. C. K. Lai and M. A. Nazaroff, “Modeling indoor particle deposition from turbulent flow onto smooth surfaces,” *Journal of Aerosol Science*, vol. 31, no. 4, pp. 463–476, 2000.
- [3] C. G. Speziale and S. Thangam, “Analysis of an RNG based turbulence model for separated flows,” *International Journal of Engineering Science*, vol. 30, no. 10, pp. 1379–1388, 1992.
- [4] E. Cunningham, “On the velocity of steady fall of spherical particles through fluid medium,” *Proceedings of the Royal Society A*, vol. 83, pp. 357–365, 1910.
- [5] G. G. Stokes, “On the effect of the internal friction of fluids on the motion of pendulums,” *Transactions of the Cambridge Philosophical Society*, vol. 9, pp. 8–106, 1851.
- [6] A. Einstein, “Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen,” *Annalen der Physik*, vol. 322, no. 8, pp. 549–560, 1905.